

GPUMODE Lecture Study Notes: Lecture 24 SoL Scanning

Core Problem the Lecture Addresses

This lecture explains how to make prefix scan run near the GPU's **speed of light**, abbreviated as **SoL**. The point is not merely to show that one implementation is faster than another weak baseline. The point is to estimate the theoretical performance limit of scan on a GPU and then ask how close a real implementation gets to that limit.

A prefix scan takes an input sequence

$$x_0, x_1, x_2, \dots, x_{n-1}$$

and produces all intermediate reductions. For inclusive sum scan,

$$y_i = \sum_{j=0}^i x_j.$$

For exclusive sum scan,

$$y_i = \sum_{j=0}^{i-1} x_j.$$

More generally, for an associative operator $\oplus$, inclusive scan is

$$y_i = x_0 \oplus x_1 \oplus \dots \oplus x_i.$$

Scan matters because it turns computations that look sequential into computations that can be parallelized. It appears in stream compaction, radix sort, dynamic allocation, graph processing, parsing, and sequence-model workloads. The lecture's central message is that scan is simple to describe, but very difficult to implement at peak GPU bandwidth.

The lecture develops three ideas:

1. how to reason about scan using speed-of-light analysis;

2. why some scan algorithms are bandwidth-bound while others are latency-bound;
3. why production scan code should usually use CUB or Thrust.

A large speedup over a poor baseline does not prove that a kernel is good. A kernel is good when it is close to its theoretical speed-of-light limit.

Required Background

Associativity and Scan

Scan requires an associative operator. Addition is the simplest example:

$$(a + b) + c = a + (b + c).$$

Associativity matters because it lets the implementation reorganize the order of computation without changing the result.

A sequential inclusive scan satisfies

$$\begin{aligned} y_0 &= x_0, \\ y_1 &= x_0 + x_1, \\ y_2 &= x_0 + x_1 + x_2, \\ y_i &= \sum_{j=0}^i x_j. \end{aligned}$$

The recurrence

$$y_i = y_{i-1} + x_i$$

looks serial. Parallel scan algorithms break this dependency by computing local partial scans and then adding prefixes from earlier tiles.

CUDA Execution Model

A CUDA kernel launches a grid of thread blocks. A thread block runs on a streaming multiprocessor, or **SM**. Threads execute in groups of 32 called **warps**. The execution hierarchy is

Grid $\rightarrow$ Thread Blocks $\rightarrow$ Warps $\rightarrow$ Threads.

The memory hierarchy is

Registers $\rightarrow$ Shared Memory / L1 $\rightarrow$ L2 Cache $\rightarrow$ Global Memory.

For scan, the local computation inside one thread block is not the main difficulty. The harder part is propagating prefix information between thread blocks without serializing the whole grid.

CUB and Thrust

CUB provides low-level GPU primitives such as device-wide scan, block scan, warp scan, cooperative block load, cooperative block store, and decoupled lookback. Thrust scan operations can use CUB internally. The practical rule is:

Implement scan by hand to learn CUDA. Use CUB or Thrust when scan is needed in production.

Main Concepts

Speed of Light

Speed of light is the theoretical best performance that an algorithm can achieve on a specific GPU. The first question is whether a kernel is compute-bound, memory-bandwidth-bound, or latency-bound.

If the kernel is compute-bound, the relevant limit is peak arithmetic throughput:

Peak FLOP/s.

If the kernel is memory-bound, the relevant limit is peak memory bandwidth:

Peak Bytes/s.

If the kernel is latency-bound, the relevant limit is the latency of a dependency chain, such as one block publishing a value and another block observing that value.

For a memory-bound kernel, the speed-of-light fraction is

$$\text{SoL Fraction} = \frac{\text{Measured Bandwidth}}{\text{Peak Memory Bandwidth}}.$$

Why Speedup Alone Is Misleading

Suppose one kernel copies data with one thread and another kernel copies data with a full thread block. The second kernel may be much faster, but it can still reach only a small fraction of peak memory bandwidth. Speedup only answers how much faster one implementation is than another. Speed-of-light analysis answers how close the implementation is to the hardware limit.

Arithmetic Intensity

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{Arithmetic Operations}}{\text{Bytes Moved}}.$$

A simple FP32 scan performs roughly one addition per element. It must read each input and write each output. Since each FP32 value is four bytes,

$$\text{Bytes Per Element} = 4 + 4 = 8.$$

Therefore,

$$\text{Arithmetic Intensity}_{\text{scan}} \approx \frac{1}{8}.$$

This is low, so a good scan implementation is usually memory-bandwidth-bound rather than compute-bound.

Roofline Model

The roofline model gives the bound

$$\text{Performance} \leq \min(\text{Peak Compute}, \text{Arithmetic Intensity} \times \text{Peak Memory Bandwidth}).$$

For scan, arithmetic intensity is low, so the bandwidth term dominates:

$$\text{Performance}_{\text{scan}} \approx \text{Memory Bandwidth Limit}.$$

Thus, an ideal scan should approach memory-copy bandwidth.

Hardware-Level Explanation

Minimum Memory Traffic

For n elements of size s bytes, scan must read each input and write each output:

$$\text{Minimum Bytes} = 2ns.$$

If peak memory bandwidth is B_{peak} , ideal runtime is

$$T_{\text{ideal}} = \frac{2ns}{B_{\text{peak}}}.$$

If measured runtime is T_{measured} , measured bandwidth is

$$B_{\text{measured}} = \frac{2ns}{T_{\text{measured}}}.$$

The achieved percentage of speed of light is

$$\text{SoL}_{\text{percent}} = 100 \cdot \frac{B_{\text{measured}}}{B_{\text{peak}}}.$$

The Inter-Block Prefix Problem

Assume each thread block processes one tile. Let

$$S_{\text{tile}} = T_{\text{block}} \cdot I_{\text{thread}} \cdot s,$$

where T_{block} is threads per block, I_{thread} is items per thread, and s is bytes per element.

Each block can scan its own tile locally. But block k also needs the sum of all previous tiles:

$$P_k = \sum_{b=0}^{k-1} \sum_{i \in \text{tile}(b)} x_i.$$

For an element inside block k , the final value is

$$y_i = \text{LocalScan}_i + P_k.$$

The main challenge is computing P_k for all blocks without forcing a strict block-by-block dependency chain.

Mathematical Reasoning

Reduce-Then-Scan

A reduce-then-scan algorithm performs extra global memory operations. If it performs about $3n$ element memory operations while the ideal scan performs $2n$, its upper bound is

$$\frac{2n}{3n} = \frac{2}{3} \approx 66.7\%.$$

This is an algorithmic limit caused by extra memory traffic.

Hierarchical Scan

A hierarchical scan may read and write the data twice, giving about $4n$ element memory operations. Compared with the ideal $2n$, the upper bound is

$$\frac{2n}{4n} = \frac{1}{2} = 50\%.$$

Even a perfect implementation of that memory-traffic pattern cannot exceed this limit.

Chained Stream Scan

A chained scan has ideal memory traffic, $2n$, but each block waits for the previous block. This creates the dependency chain

$$\text{Block 0} \rightarrow \text{Block 1} \rightarrow \text{Block 2} \rightarrow \cdots$$

Let L_{msg} be message-passing latency in cycles and let f be GPU frequency in cycles per second. The number of serialized messages per second is approximately

$$R_{\text{msg}} = \frac{f}{L_{\text{msg}}}.$$

Each message allows one tile to be processed. Since scan reads and writes the tile, the latency-limited bandwidth is

$$B_{\text{chain}} = \frac{2S_{\text{tile}}f}{L_{\text{msg}}}.$$

The corresponding speed-of-light fraction is

$$\text{SoL}_{\text{chain}} = \frac{B_{\text{chain}}}{B_{\text{peak}}} = \frac{2S_{\text{tile}}f}{L_{\text{msg}}B_{\text{peak}}}.$$

Increasing tile size helps because S_{tile} is in the numerator. However, larger tiles increase register pressure and shared memory usage.

Code Walkthrough

Block-Local Scan

The following simplified kernel scans only within one block. It is educational but not a complete device-wide scan.

```
template <int BLOCK_THREADS>
__global__ void local_scan_kernel(const int* input,
                                int* output,
                                int n) {
    __shared__ int temp[BLOCK_THREADS];

    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + threadIdx.x;

    int x = 0;
    if (gid < n) {
        x = input[gid];
    }

    temp[tid] = x;
    __syncthreads();

    for (int offset = 1; offset < BLOCK_THREADS; offset *= 2) {
        int y = 0;
        if (tid >= offset) {
            y = temp[tid - offset];
```

```

    }
    __syncthreads();
    if (tid >= offset) {
        temp[tid] += y;
    }
    __syncthreads();
}

if (gid < n) {
    output[gid] = temp[tid];
}
}

```

The shared memory array stores intermediate values. Each iteration doubles the prefix distance. Synchronization is required because threads communicate through shared memory. The weakness is that each block computes only a local prefix.

Buggy Chained Scan

The following pattern is unsafe because another block can observe the flag before the aggregate is visible.

```

struct TileState {
    int aggregate;
    int flag;
};

__device__ TileState* states;

__global__ void buggy_chain_scan(const int* input,
                                int* output,
                                int n) {
    int tile = blockIdx.x;
    int local_aggregate = compute_tile_scan(input, output, n);

    if (tile > 0) {
        while (states[tile - 1].flag == 0) {
            /* spin */
        }
        int prefix = states[tile - 1].aggregate;
        add_prefix_to_tile(output, prefix, n);
        local_aggregate += prefix;
    }

    states[tile].aggregate = local_aggregate;
    states[tile].flag = 1;
}

```

```
}
```

The problem is memory ordering. Correct producer-consumer communication requires acquire-release ordering or a design that stores the flag and message together.

Packing Flag and Aggregate

If the aggregate is 32 bits, a 64-bit word can hold both the flag and aggregate:

Packed State = (flag, aggregate).

```
__device__ unsigned long long pack_state(unsigned int flag,
                                         unsigned int aggregate) {
    return (static_cast<unsigned long long>(flag) << 32) |
           static_cast<unsigned long long>(aggregate);
}

__device__ void unpack_state(unsigned long long state,
                             unsigned int& flag,
                             unsigned int& aggregate) {
    flag = static_cast<unsigned int>(state >> 32);
    aggregate = static_cast<unsigned int>(state & 0xffffffffu);
}
```

This reduces the critical communication path to one producer store and one consumer load. It also removes unnecessary fences when the state fits in one architectural word.

CUB-Style Block Load, Scan, and Store

CUB lets a thread block cooperatively load a tile, scan it, and store it. The following code is representative rather than a full production kernel.

```
#include <cub/cub.cuh>

template <int BLOCK_THREADS, int ITEMS_PER_THREAD>
__global__ void cub_block_scan_kernel(const int* input,
                                     int* output,
                                     int n) {
    using BlockLoad =
        cub::BlockLoad<int, BLOCK_THREADS, ITEMS_PER_THREAD>;
    using BlockScan =
        cub::BlockScan<int, BLOCK_THREADS>;
    using BlockStore =
        cub::BlockStore<int, BLOCK_THREADS, ITEMS_PER_THREAD>;

    __shared__ typename BlockLoad::TempStorage load_storage;
```

```

__shared__ typename BlockScan::TempStorage scan_storage;
__shared__ typename BlockStore::TempStorage store_storage;

int tile_offset =
    blockIdx.x * BLOCK_THREADS * ITEMS_PER_THREAD;

int items[ITEMS_PER_THREAD];

BlockLoad(load_storage).Load(input + tile_offset, items);
__syncthreads();

BlockScan(scan_storage).InclusiveSum(items, items);
__syncthreads();

BlockStore(store_storage).Store(output + tile_offset, items);
}

```

The block load pulls data from global memory into registers. The block scan computes local prefixes. The block store writes results back to global memory. The tile contains

$$\text{Tile Elements} = \text{Block Threads} \cdot \text{Items Per Thread}.$$

Coalesced Loading and Transpose

Naive striped loads can waste memory bandwidth because memory is fetched in sectors. If many bytes in each fetched sector are unused, useful bandwidth is much lower than raw bandwidth. CUB can use transposed load and store strategies to keep global memory access coalesced while arranging data conveniently in registers.

The trade-off is

better memory coalescing versus higher shared memory usage.

Decoupled Lookback

Motivation

Chained scan is latency-bound because block k waits for block $k - 1$. Decoupled lookback avoids this strict dependency by allowing a block to inspect several previous tile states and combine partial results locally.

Let A_k be the aggregate of tile k . Let P_k be the inclusive prefix through tile k :

$$P_k = \sum_{b=0}^k A_b.$$

Block k needs the exclusive tile prefix

$$E_k = \sum_{b=0}^{k-1} A_b.$$

If block k finds an earlier inclusive prefix P_j and accumulates partial aggregates after it, then

$$E_k = P_j + \sum_{b=j+1}^{k-1} A_b.$$

Why It Avoids Serialization

In chained scan, block 3 waits for block 2, and block 2 may be waiting for block 1. In decoupled lookback, block 3 can inspect block 2, block 1, and earlier blocks. If block 1 has already published an inclusive prefix and block 2 has published a partial aggregate, block 3 can compute its own prefix without waiting for block 2 to publish a full inclusive prefix.

Thus, prefix propagation is no longer a strict serial chain.

Lookback Window and Backoff

A practical implementation may load a window of previous tile states. This improves memory access efficiency, but neighboring blocks have overlapping windows, so many blocks may read the same states. That causes contention. A tuned implementation therefore uses backoff after repeatedly reading invalid states.

A conceptual loop is:

```
while (!prefix_found) {
    TileState window[LOOKBACK_WINDOW];
    load_previous_tile_states(window);

    for (int i = 0; i < LOOKBACK_WINDOW; ++i) {
        if (window[i].is_inclusive()) {
            accumulator += window[i].prefix;
            prefix_found = true;
            break;
        }
        if (window[i].is_partial()) {
            accumulator += window[i].aggregate;
        }
    }

    if (!prefix_found) {
```

```

        backoff_delay();
    }
}

```

The best window size and backoff policy depend on the GPU, data type, tile shape, and workload. This is one of the reasons CUB uses extensive tuning.

Performance Intuition

Why Scan Should Approach Memory Copy

An ideal scan reads each input once and writes each output once:

$$\text{Traffic}_{\text{scan}} = 2ns.$$

A memory copy also reads and writes:

$$\text{Traffic}_{\text{copy}} = 2ns.$$

Since scan performs little arithmetic per byte, a high-quality scan should achieve bandwidth close to memory copy bandwidth.

Why Extra Passes Are Expensive

If an algorithm performs $3n$ memory operations, its best possible fraction of the ideal $2n$ algorithm is

$$\frac{2}{3}.$$

If it performs $4n$ memory operations, its best possible fraction is

$$\frac{1}{2}.$$

For memory-bound algorithms, every extra global memory pass lowers the speed-of-light ceiling.

Why Chained Scan Is Slow

Chained scan has minimal memory traffic but creates a dependency chain. GPUs are designed for throughput, not low-latency serialized communication. If every block waits for the previous block, the hardware cannot hide latency with parallelism.

Why More Items Per Thread Helps

Increasing items per thread increases tile size:

$$S_{\text{tile}} = T_{\text{block}} \cdot I_{\text{thread}} \cdot s.$$

For chained scan,

$$B_{\text{chain}} = \frac{2S_{\text{tile}}f}{L_{\text{msg}}}.$$

A larger tile amortizes message-passing latency over more data. The limit is resource usage: registers, shared memory, and occupancy.

Why Coalescing Matters

Global memory is fetched in sectors. If a warp fetches many sectors but uses only a small fraction of each, effective bandwidth collapses:

$$B_{\text{effective}} = B_{\text{raw}} \cdot \frac{\text{Useful Bytes}}{\text{Fetched Bytes}}.$$

A coalesced layout maximizes useful bytes per transaction.

Why CUB Wins

CUB combines efficient local scan, coalesced block load and store, packed tile-state communication, decoupled lookback, tuned lookback windows, tuned backoff, and architecture-specific parameter choices. The tuning space includes block size, items per thread, load algorithm, store algorithm, cache modifiers, data type, and GPU architecture.

Common Misconceptions

A Large Speedup Means the Kernel Is Good

A kernel can be much faster than a bad baseline and still be far from peak hardware performance. Speed-of-light percentage is more meaningful than raw speedup.

Scan Is Compute-Bound

Scan performs arithmetic, but it does not perform much arithmetic per byte. For FP32 scan,

$$\text{Arithmetic Intensity} \approx \frac{1}{8}.$$

Therefore, an optimized scan is usually memory-bandwidth-bound.

Minimal Memory Traffic Guarantees High Performance

Chained scan has minimal global memory traffic, but it is latency-bound because it serializes thread blocks.

Volatile Is Enough for Inter-Block Communication

Volatile does not provide the required memory ordering. Correct producer-consumer communication between blocks requires acquire-release semantics or a safe single-word state design.

More Items Per Thread Always Helps

More items per thread increases tile size and amortizes latency, but it also increases register pressure and shared memory use. Too much per-thread work can reduce occupancy.

Practical Takeaways

1. Use speed-of-light analysis instead of relying only on speedup.
2. Estimate arithmetic intensity before optimizing.
3. For scan, ideal memory traffic is $2n$ element transfers.
4. A high-quality scan should approach memory-copy bandwidth.
5. Extra global memory passes impose hard performance ceilings.
6. Chained scan is limited by message-passing latency.
7. Packing flag and aggregate into one word improves inter-block communication.
8. Coalesced loads and stores are essential.
9. Decoupled lookback breaks strict block-by-block serialization.
10. Use CUB or Thrust for production scan.

Project or Experiment Ideas

Compare Scan Against Memory Copy

Benchmark memory copy, a custom copy kernel, CUB device scan, and Thrust inclusive scan. For each implementation, compute

$$B_{\text{measured}} = \frac{2ns}{T_{\text{measured}}}$$

and

$$\text{SoL}_{\text{percent}} = 100 \cdot \frac{B_{\text{measured}}}{B_{\text{peak}}}.$$

Sweep Items Per Thread

Run experiments with

$$I_{\text{thread}} \in \{1, 2, 4, 8, 16, 24, 32\}.$$

Measure runtime, achieved bandwidth, registers per thread, shared memory per block, and occupancy.

Study Coalesced Versus Strided Loads

Compare direct, striped, transposed, and vectorized block loads. Use Nsight Compute to inspect global load efficiency and memory-sector utilization.

Profile With Nsight Compute

For each scan kernel, inspect achieved memory throughput, percentage of peak bandwidth, warp stall reasons, L2 throughput, global memory sector counts, shared memory usage, and occupancy. Connect the counters back to the speed-of-light model.

Visual Concept

Speed-of-Light Roofline

Draw a roofline plot with arithmetic intensity on the horizontal axis and performance on the vertical axis. Place memory copy and scan in the low-arithmetic-intensity memory-bound region. Place matrix multiplication in the high-arithmetic-intensity compute-bound region.

Chained Scan Serialization

Draw thread blocks in a row. Show block 0 publishing a prefix, block 1 waiting, block 2 waiting, and so on. Label each arrow as message-passing latency.

Decoupled Lookback

Draw a later block looking backward over a window of prior tile states. Mark some states as partial aggregates and one state as an inclusive prefix. Show that the later block combines these values locally instead of waiting for a strict predecessor chain.

CUB Cooperative Tile Processing

Draw global memory feeding into block load, per-thread registers feeding into block scan, shared memory used for cooperation, and block store writing results back to global memory.

Final Mental Model

Scan appears sequential because every output depends on previous inputs. GPU scan becomes fast by splitting the work into local tile scans and inter-tile prefix propagation.

The ideal scan reads every input once and writes every output once:

$$\text{Ideal Traffic} = 2ns.$$

Therefore, an optimal scan should approach memory-copy bandwidth.

Different algorithms fail for different reasons. Reduce-then-scan and hierarchical scan move too much data. Chained scan moves the right amount of data but serializes blocks and becomes latency-bound. Decoupled lookback avoids strict serialization by allowing blocks to look backward, collect partial aggregates, find an earlier inclusive prefix, and compute their own exclusive prefix locally.

The final production mental model is

Fast Scan = minimal memory traffic
+ coalesced memory access
+ efficient local scan
+ decoupled inter-block prefix propagation
+ architecture-specific tuning.

Implement scan by hand to understand GPU programming. Use CUB when correctness, portability, and speed-of-light performance matter.